Semantic Retrieval for Renewable Energy Research

A Synthetic Sample Paper for Document Ingestion and Vector Search

Document type: Demonstration research paper **Version:** 1.0

Abstract

This synthetic paper is designed to test a document-processing pipeline for the OrchestrAI prototype. It describes how a retrieval system can extract text from research documents, divide the text into overlapping chunks, convert chunks into dense vector embeddings, and retrieve relevant passages for a user question. The example domain is renewable energy storage. It is not a peer-reviewed publication and contains no empirical experimental results. A working retrieval system should return passages that discuss the question asked, while preserving page numbers and document metadata for traceability.

1. Introduction

Researchers often need to compare information across many technical papers, reports, and project documents. Manual reading can be slow when the source collection is large. A document retrieval pipeline can make this process more efficient by extracting searchable text and locating passages that are semantically related to a question, even when the wording of the question differs from the wording in the source.

For example, a question such as “How can batteries improve solar power reliability at night?” may be related to a passage about storing electricity generated during daylight hours. Keyword-only retrieval may miss this connection if the same terms are not present. Semantic retrieval represents text and queries as vectors in a shared embedding space and ranks passages by vector similarity.

2. Background and Concepts

2.1 Text extraction and preprocessing

A PDF ingestion system first extracts text from each page. It should preserve the page number associated with each extracted passage. The system can normalize repeated whitespace and remove empty pages, but it should avoid aggressive transformations that remove technical symbols, abbreviations, or meaningful punctuation. Scanned PDFs may contain only page images; those files require optical character recognition (OCR), which is outside the scope of this sample.

2.2 Chunking with overlap

Long documents are typically split into smaller text chunks before embedding. Chunking helps keep the text sent to an embedding model within practical limits and makes retrieved results more focused. A small overlap between consecutive chunks can preserve context near boundaries. The chunk size and overlap should be configurable and evaluated on representative documents rather than treated as universal constants.

2.3 Embeddings and vector databases

An embedding model maps a text segment to a numeric vector. In this prototype, the selected model is sentence-transformers/all-MiniLM-L6-v2. A vector database such as Qdrant can store vectors alongside payload fields, including document identifier, filename, page number, chunk index, and the original chunk text. Cosine similarity can then be used to retrieve the most relevant stored vectors for a query.

3. Proposed Processing Pipeline

The demonstration workflow consists of six stages: (1) load a local PDF, (2) extract page-level text, (3) clean whitespace and ignore empty pages, (4) split text into overlapping chunks, (5) generate embeddings, and (6) store vectors and metadata in Qdrant. At query time, the system embeds the user question, searches the vector collection, and returns the highest-ranked chunks with their source metadata.

Stage	Input	Output
PDF ingestion	PDF file	Text by page

Stage	Input	Output
Preprocessing	Extracted text	Clean text
Chunking	Clean text	Overlapping chunks
Embedding	Text chunks	Dense vectors
Vector storage	Vectors + metadata	Qdrant points
Semantic search	Question vector	Ranked source passages

4. Example Research Domain: Solar Energy Storage

Solar panels generate electricity when sunlight is available, but demand may continue after sunset. Battery storage can retain some of the electricity generated during the day and release it later. This can improve the dispatchability of solar power and help balance supply and demand. Battery systems also introduce practical considerations, including capital cost, degradation, operating temperature, charging efficiency, recycling, and the sourcing of raw materials.

A semantic search query such as “What makes renewable electricity more reliable after dark?” should retrieve passages about battery storage, retained daytime electricity, and nighttime supply. A query about document processing should instead retrieve passages about extraction, chunking, embeddings, and metadata.

5. Evaluation Checklist

A basic functional evaluation should verify that the system can (a) open a valid text-based PDF, (b) extract non-empty text with correct page numbers, (c) create chunks with the configured overlap, (d) generate vectors with consistent dimensions, (e) store vectors and metadata without deleting existing collections, and (f) retrieve relevant passages for several natural-language questions. Results should be inspected manually. A non-zero similarity score alone does not prove that retrieval is accurate.

6. Limitations

This document is synthetic and exists only as a safe test fixture. It does not report original experiments, validated measurements, or peer-reviewed findings. Its short length and simple vocabulary do not represent the complexity of a real scientific corpus. Production evaluation should include diverse documents, tables, references, multiple writing styles, and questions whose answers are known in advance.

7. Conclusion

A PDF-to-vector retrieval pipeline connects document processing with semantic search. By retaining source text and page metadata, it can return passages that are both relevant to a question and traceable to the document. This sample paper provides a repeatable starting point for testing the OrchestrAI Review 2 prototype before indexing real research literature.

References

[1] Reimers, N. and Gurevych, I. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” Proceedings of EMNLP-IJCNLP, 2019.

[2] Qdrant. “Qdrant Documentation.” Documentation for vector search and payload storage.

[3] Sentence Transformers. “Pretrained Models and Usage Documentation.”

Note: This is an original synthetic demonstration document generated for software testing. It must not be cited or presented as a real scholarly publication.